

Исследование возможностей GitHub Desktop для работы с Git

Введение

Git — распределённая система управления версиями, которая широко используется при разработке программного обеспечения;

GitHub Desktop — это бесплатное кроссплатформенное графическое приложение, созданное для упрощения работы с Git. Оно обладает рядом особенностей, например:

- Кроссплатформенность: GitHub Desktop прекрасно работает и на Windows, и на macOS, даже интерфейс практически полностью одинаковый. На Linux официально не поддерживается.
- Упрощённый интерфейс: GitHub Desktop обладает понятным пользовательским интерфейсом, благодаря которому несведущие в Git пользователи смогут быстрее освоиться и начать работать с командами без использования командной строки.

Основные возможности и соответствующие им команды

Инициализация нового локального репозитория (git init)

Итак, для создания репозитория необходимо открыть GitHub Desktop, в левом верхнем углу найти и нажать на «File» и в появившемся меню выбрать «New repository» (Рис. 1).

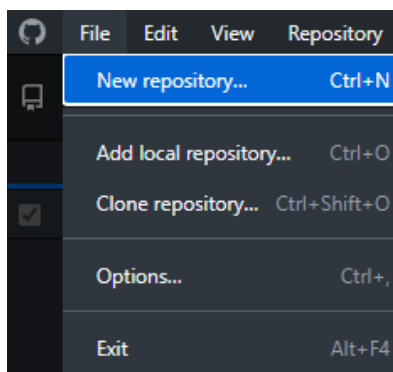
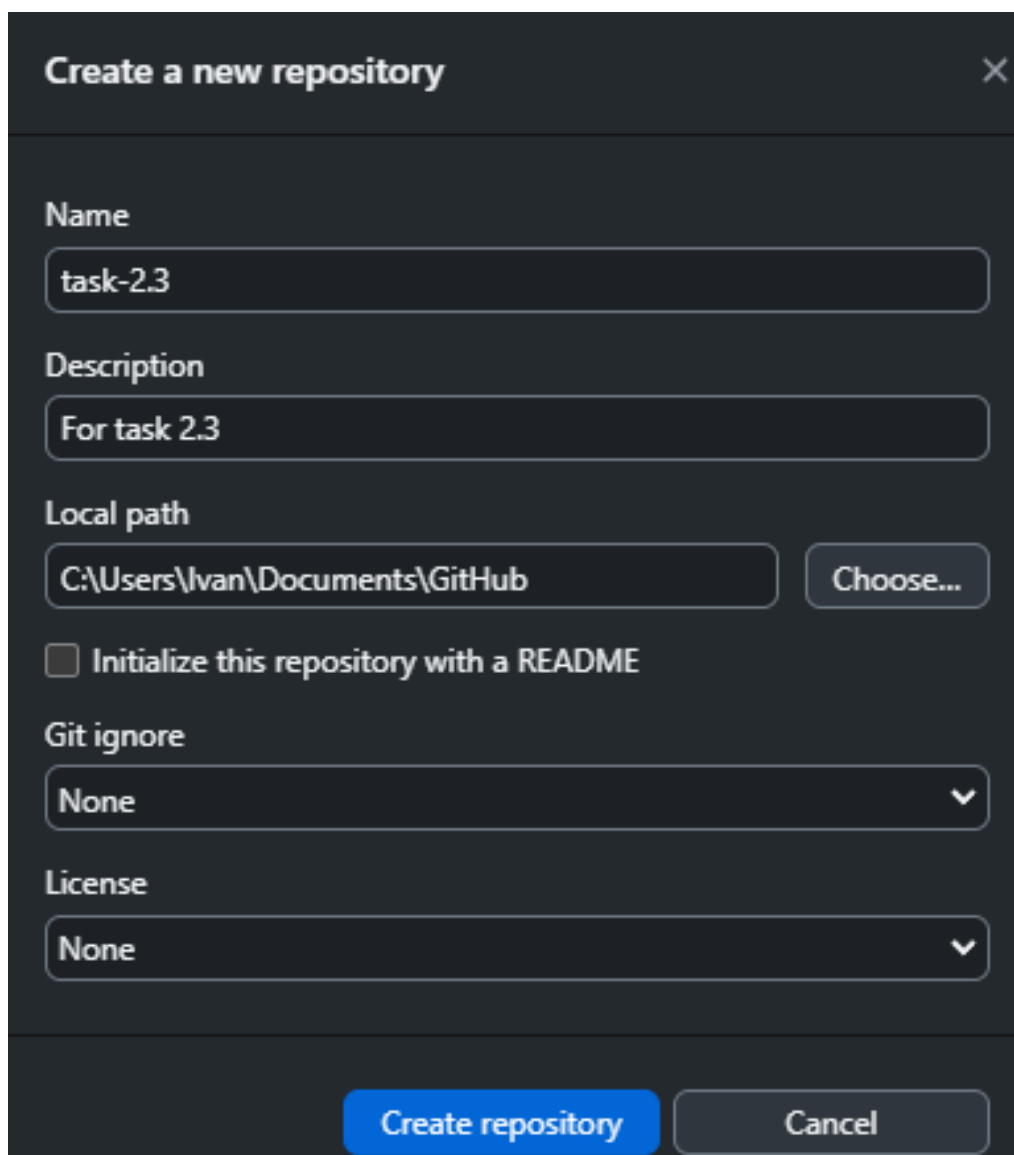


Рисунок 1 – Создание нового репозитория

В появившемся окне необходимо указать имя репозитория, добавить ему описание, указать путь, при желании поставить галочку около инициализации репозитория файлом README и при необходимости добавить файл gitignore и лицензию, после чего нажать на «Create repository» (Рис. 2).



Create a new repository [X]

Name
task-2.3

Description
For task 2.3

Local path
C:\Users\Ivan\Documents\GitHub [Choose...]

☐ Initialize this repository with a README

Git ignore
None [v]

License
None [v]

[Create repository] [Cancel]

Рисунок 2 – Окно создания нового репозитория

Аналогичная команда в терминале: `git init <имя_репозитория>` – команда для инициализации репозитория с указанным именем.

Добавление изменений и коммит (git add и git commit соответственно)

Допустим, мы добавили в репозиторий 1 файл и изменили его. Теперь он будет отображаться в панели изменений слева. Таким образом, для коммита необходимо выбрать все желаемые файлы, поставив слева от них галочку, указать заголовок и описание коммита и нажать на «commit to [current branch]», в моём примере это «commit to main» (Рис. 3).

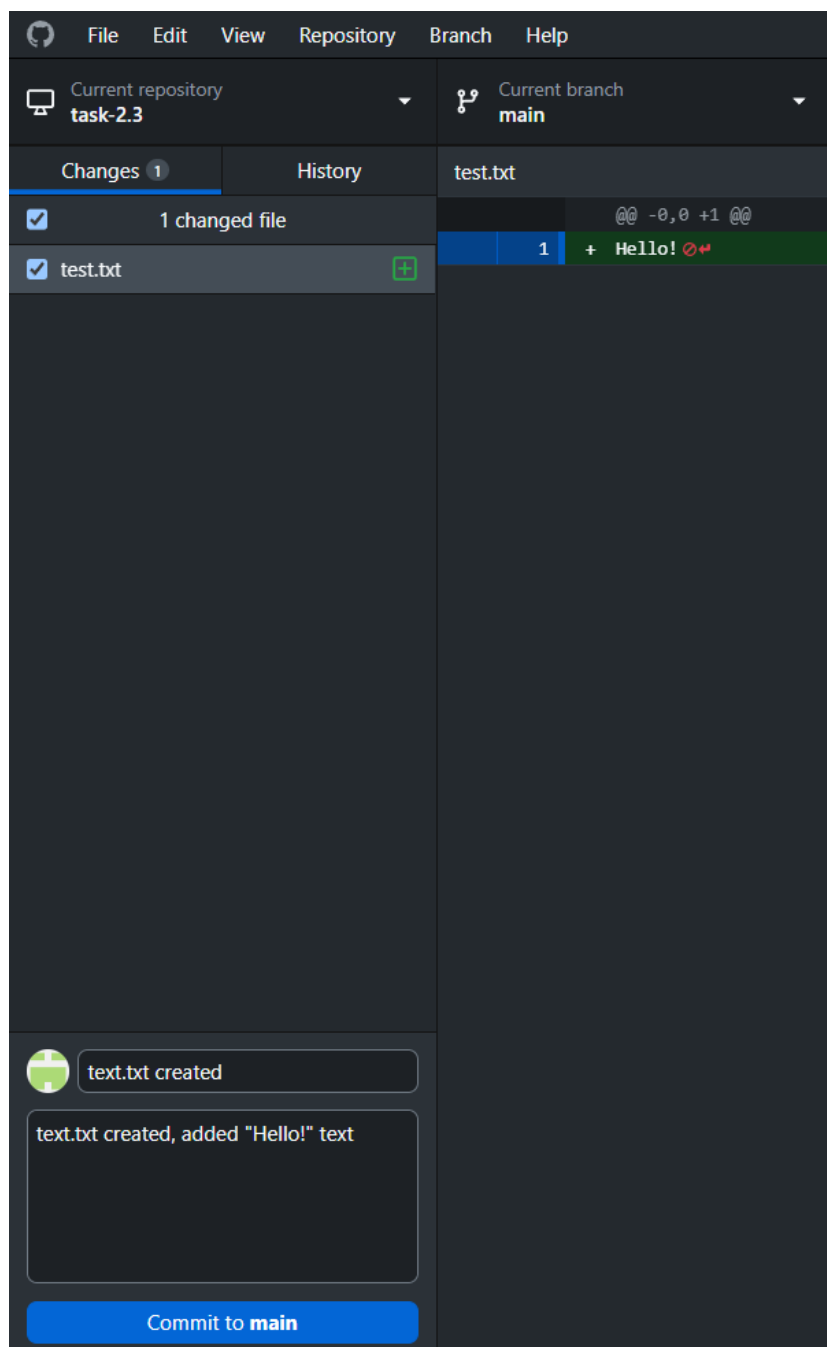


Рисунок 3 – Создание коммита

Отправка изменений на удалённый репозиторий (git push)

После внесения изменений в локальный репозиторий появится возможность отправить их на удалённый репозиторий. Кнопка «Push origin», находящаяся около выбранных репозитория и ветки, станет активной – при нажатии на неё и произойдёт отправка изменений (или пуш) на удалённый репозиторий (Рис. 4, 5).

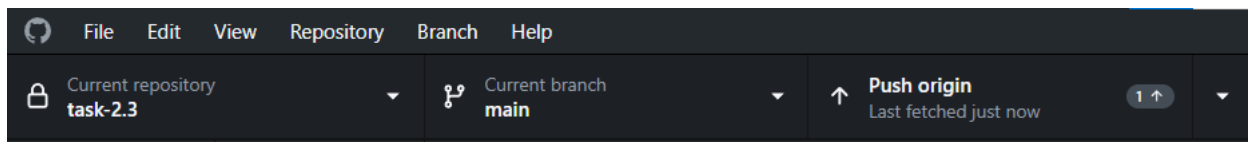


Рисунок 4 – Кнопка «Push origin»

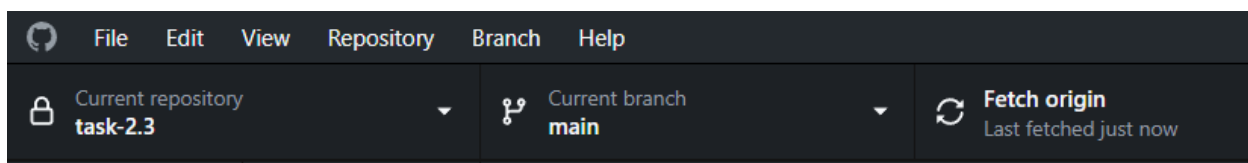


Рисунок 5 – После нажатия на кнопку «Push origin»

Аналогичная команда в терминале:

```
git push <имя_удалённого_репозитория> <имя_отправляемой_ветки>
```

Зачастую она выглядит так: `git push origin main`

Получение обновлений из удалённого репозитория (git pull)

Что ж, отправить изменения на удалённый репозиторий мы можем, а что, если кто-то другой вносил изменения? В таком случае (когда в ветке удалённого репозитория есть новые коммиты), при нажатии на кнопку «Fetch origin» (для которой тоже существует аналогичная команда `git fetch`), она заменится на «Pull origin» (Рис. 6), и уже нажатием на «Pull origin» можно будет получить изменения из удалённого репозитория. После этого кнопку «Pull origin» снова заменит кнопка «Fetch origin».

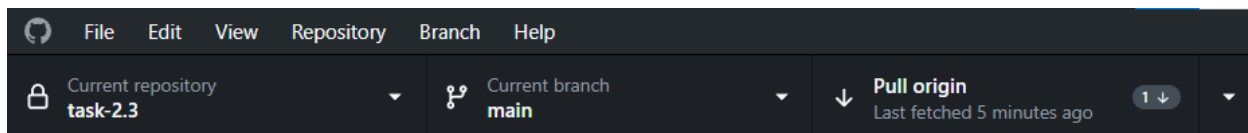


Рисунок 6 – Кнопка «Pull origin»

Аналогичная команда в терминале:

```
git pull <имя_удалённого_репозитория> <имя_получаемой_ветки>
```

Зачастую она выглядит так: `git pull origin main`

Работа с ветками (git branch, git switch, git checkout, git merge)

С внесением изменений разобрались, с получением изменений тоже, теперь поговорим о работе с ветками. Начнём с создания: процесс практически не отличается от создания репозитория вручную, только теперь нужно открыть меню «Current branch» и там выбрать «New branch» (Рис. 7), в открывшемся окне остаётся только дать имя новой ветке и нажать «Create branch» (Рис. 8). Переключаться между ветками можно в том же меню «Current branch» – нужно просто выбрать желаемую ветку в списке ниже (Рис. 9).

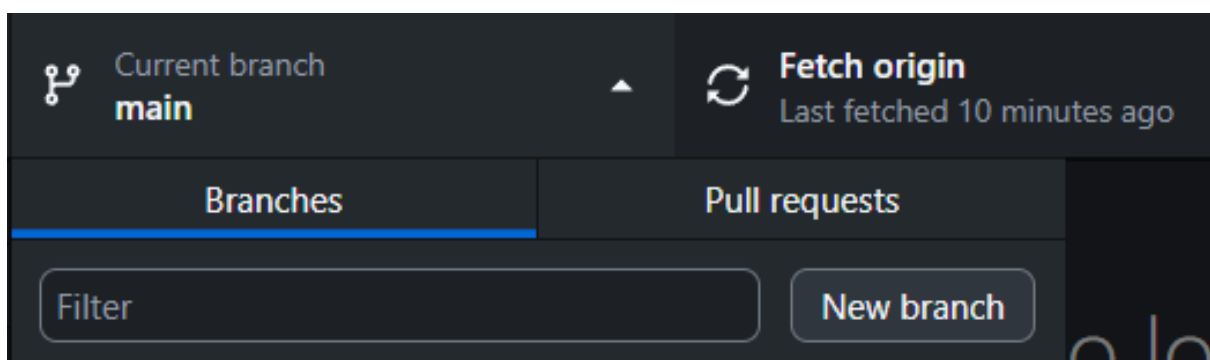


Рисунок 7 – Создание новой ветки

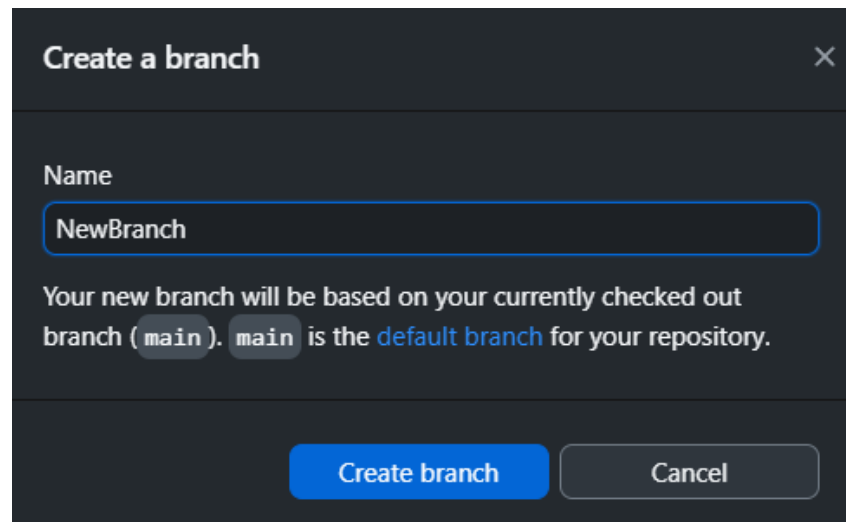


Рисунок 8 – Окно создания новой ветки

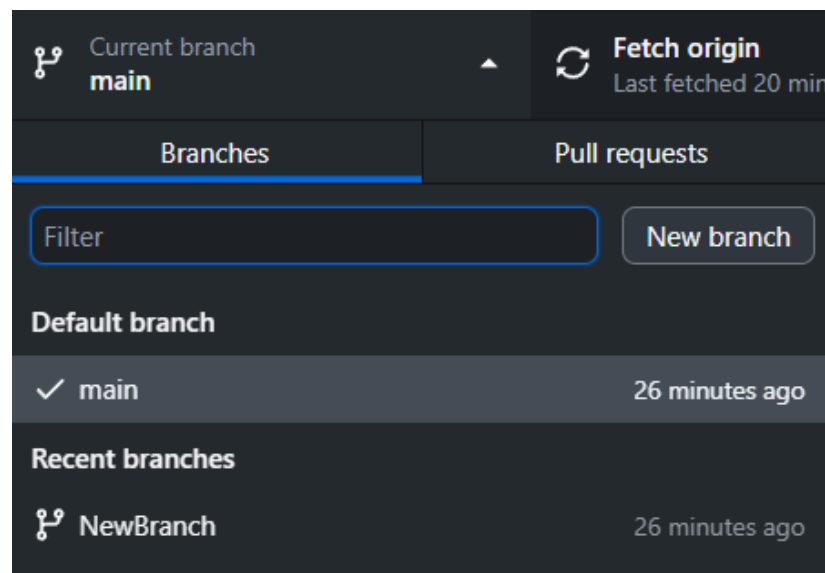


Рисунок 9 – Переключение между ветками

Аналогичная команда для создания новой ветки: `git branch <имя_ветки>`. В моём примере будет «`git branch NewBranch`».

Аналогичная команда для переключения между ветками: `git switch <имя_ветки>`. В моём примере будет «`git switch NewBranch`».

Есть ещё команда `git checkout`, которая уже заметно интереснее. Дело в том, что в целом эта команда может заменить обе верхние (так делать можно, но не рекомендуется), а помимо этого ещё имеет и своё предназначение. Рассмотрим:

- `git checkout <имя_ветки>` – аналог `git switch` (переход к указанной ветке);
- `git checkout -b <имя_ветки>` – аналог `git branch` (создание ветки с указанным именем и переход к ней)
- `git checkout -- <путь_к_файлу>` – отмена изменений в указанном файле или его возвращение к состоянию последнего коммита (а, например, команда «`git checkout -- .`» делает то же самое, но для всех файлов в каталоге).

Рассмотрим также команду `git merge`. Формат команды в терминале: `git merge <имя_ветки_которую_хотим_слить_с_текущей>`. Но на самом деле, если задуматься, мы уже рассматривали команду `git merge`, когда использовали `git pull`: по сути, команда `git pull` – это последовательное использование команд `git fetch` и `git merge`, только `git pull` мы использовали для слияния ветки из удалённого репозитория с текущей веткой локального репозитория, а `git merge` можно использовать, например, для слияния 2 веток в локальном репозитории.

Также затронем разрешение конфликтов при слиянии веток. Бывает, что при попытке слияния веток появляется сообщение «There are conflicts that must be resolved». В таком случае необходимо нажать на «View file in editor» и вручную изменить файл в открывшемся редакторе с пометками «<<<<<<<<», «=====», «>>>>>>>>». После этого файл нужно сохранить и отметить как «resolved», а дальше сделать коммит для завершения слияния.

Закончим с ветками на пулл-реквестах (pull request). Если бы я работал не в своём репозитории, то создал бы свою копию этого репозитория, вносил изменения локально, а потом, вместо пуша, отправил бы именно пулл-реквест. То есть, я бы не внёс изменения в изначальный репозиторий (push), а лишь предложил бы внести свои изменения в общий репозиторий (pull-request), но вот будут они приняты или нет – вопрос уже другой. Теперь о том, как его сделать: в верхнем меню необходимо найти Branch, а в открывшемся меню нажать на «Create pull request», после чего произойдёт переход в браузер.

Клонирование репозитория

Рассмотрим, как можно создать копию существующего репозитория. Для этого в верхнем меню необходимо найти «File» и в появившемся окне нажать на «Clone repository...» (Рис. 10). Далее надо будет выбрать репозиторий, копию которого мы хотим создать. Если это наш репозиторий, то можно просто выбрать его в списке (если он удалённый) или указать путь (если он локальный), но можно прибегнуть и к универсальному способу – использовать ссылку на репозиторий (Рис. 11).

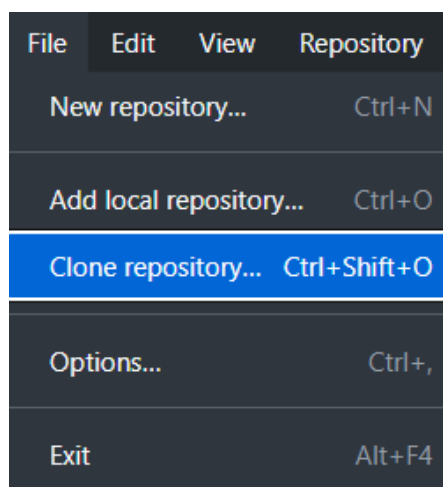


Рисунок 10 – Клонирование репозитория

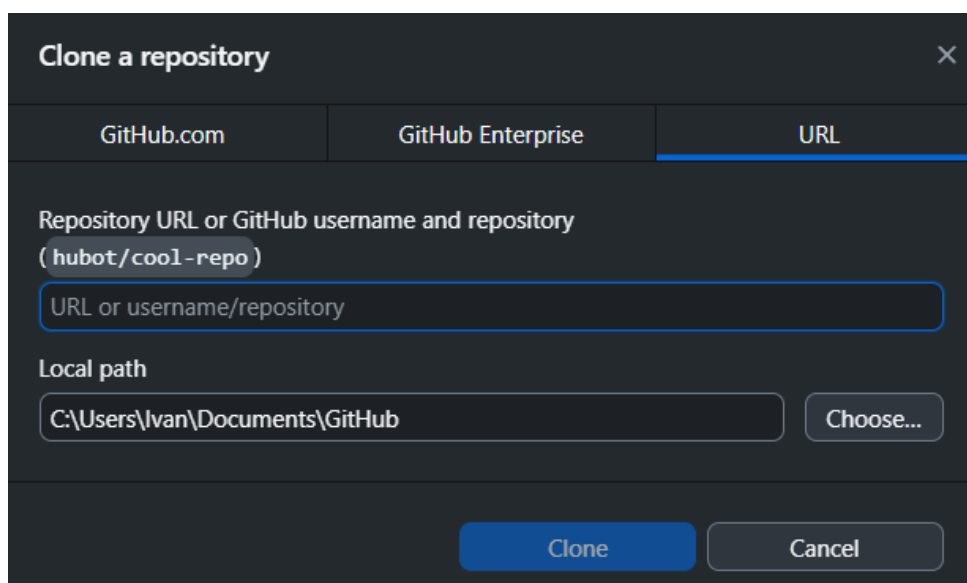


Рисунок 11 – Окно клонирования репозитория

Вывод

GitHub Desktop – это отличное решение для любого человека, который так или иначе пользуется GitHub (обычный студент, у которого там хранится портфолио, или опытный разработчик из сильной команды – не имеет значения). Он позволяет весьма легко и удобно выполнять базовые операции: инициализацию, коммиты, пуши, пулы, а также создание веток, переключение между ними, их слияние и, немаловажно, визуальный интерфейс для разрешения конфликтов при слиянии.